

# Digital Twin Design Document

Letizia Laura, Jordan Cummings, Henry Hong

May 2026

## 1 Introduction

The Digital Twin (DT) is a simulated model of the Seaglider’s dynamic behavior that replicates its motion and response throughout a dive. Using actuator states from the Variable Buoyancy Device (VBD), battery roll and pitch systems, mission data, and environmental parameters, the DT computes vehicle forces, moments, velocities, and motion states. The model also incorporates vehicle geometry, inertial and mass properties, and hydrodynamic coefficients that define fluid interaction. Within the D.A.P. system, the DT generates simulated estimates of the Seaglider’s orientation, angular rates, and position, allowing comparison with recorded mission data to identify faults or performance deviations.

## 2 Digital Twin Design

### 2.1 D.A.P Inputs

To run the DT function the user must upload the Seaglider mission ‘.log’ and ‘.nc’ files collected during a dive in the D.A.P main function. These files are passed into the **Log\_File\_Unpacker** and **NC\_File\_Unpacker** functions, where the raw mission data is extracted and unnecessary data fields are removed.

The processed mission data is then passed into the **Create\_Mission\_Data\_Matrix** function, which compiles the required mission parameters into the Mission Data Matrix,  $M$ . The matrix  $M$  contains the actuator states, vehicle states, and science parameters required by the Digital Twin simulation.

$$M = \begin{bmatrix} \text{elaps\_t\_0000} & \text{elaps\_t} & \text{depth} & \text{head} & \text{pitchAng} & \text{rollAng} & \text{pitchCtl} & \text{rollCtl} & \text{vbdCC} & \text{temp} \\ \text{RHO} & \text{\$MHEAD} & & & & & & & & \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & & & & & & & & \end{bmatrix}$$

At times the .nc files columns may contain missing values represented as ‘NaN’s. This is not necessarily an error as pilots often set discontinuous sampling rates for some science parameters based on a mission’s needs. To prevent interruptions in the simulation, the Matrix Builder includes a filler feature that replaces missing data values. If a column begins

with one or more ‘NaN’ values, those entries are replaced using the first valid value that appears later in the column. If ‘NaN’ values appear anywhere else within the column, they are replaced using the most recent valid value previously recorded in the data set. This process ensures that all required parameters remain continuous throughout the simulation.

## 2.2 Digital Twin Function

### 2.2.1 Inputs to the Plant

The Digital Twin function takes three primary inputs:

- Mission data matrix  $M$
- Parameter file
- Hydrodynamic coefficient file

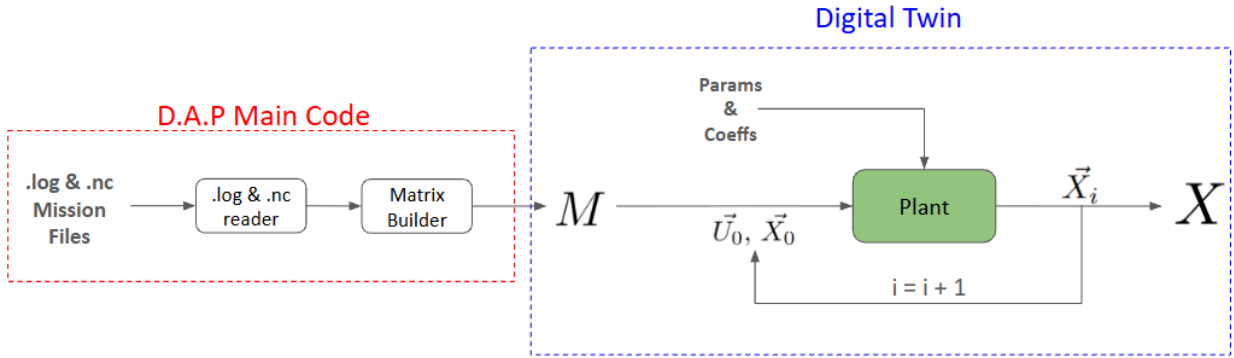


Figure 1: Digital Twin final workflow

The mission data matrix  $M$  is further processed to generate the actuator state vector  $\vec{U}_i$  and the vehicle state vector  $\vec{X}_i$ . The actuator state vector contains the commanded actuator positions for a single timestamp recorded in the mission data. The format of the vector is as follows

$$\vec{U}_i = \begin{bmatrix} x_{bat} \\ \phi_{bat} \\ vbdCC \end{bmatrix}$$

where

- $x_{bat}$  : battery position along  $b_1$ -axis in body frame
- $\phi_{bat}$  : battery rotation angle in the  $yz$ -plane
- $vbdCC$  : volume of oil in Variable Buoyancy Device (VBD) in cc.

The vehicle state vector contains the simulated dynamic state of the Seaglider. The format of  $\vec{X}_i$  is outlined below

$$\vec{X}_i = \begin{bmatrix} \phi \\ \theta \\ heading \\ \dot{\phi} \\ \dot{\theta} \\ heading \\ u \\ v \\ w \\ x \\ y \\ z \end{bmatrix}$$

where

- $\phi, \theta, heading$  : the three Euler angles measured by the attitude sensor
- $\dot{\phi}, \dot{\theta}, heading$  : the roll rate, pitch rate, and heading rate in body frame
- $u_i, v_i, w$  : translational velocities WRT the current heading
- $x, y, z$  : positions in the glide-path frame

The simulation begins by initializing the Plant model using the vectors  $\vec{U}_{i=0}$  and  $\vec{X}_{i=0}$ . The notation  $i = 0$  denotes the first timestep in the simulation. Since the Seaglider records data while floating at the surface prior to beginning its dive, the initial values are not taken directly from the first row of the mission data matrix. Instead, the initialization values are extracted from the first row in the mission data where depth is greater than an arbitrary distance denoted in the code as **simStartDepth.cm**. The values for both  $\vec{U}_{i=0}$  and  $\vec{X}_{i=0}$  are initialized from this first simulation row.

The initial vehicle state vector  $\vec{X}_{i=0}$  is used one time only to initialize the Plant before the simulation loop begins. The initial state vector contains the starting dynamic state of the Seaglider and everything but the attitudes and vertical velocity is set to zero.

$$\vec{X}_{i=0} = \begin{bmatrix} \phi \\ \theta \\ heading \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ w \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

The **parameter** and **coefficient** files define the physical and hydrodynamic characteristics of the Seaglider configuration being simulated by the D.A.P. system. When the Digital Twin is executed for a specific damage scenario, these files determine the vehicle properties and hydrodynamic behavior that govern its dynamic response and are used directly by the dynamics code during each Plant iteration.

The **parameter** file contains constants and geometric properties associated with each nominal and fault configuration, including:

- Geometry of the fault case
- Battery actuator properties
- Mass and inertia matrices
- Environmental properties

The **coefficient** file contains lookup tables of hydrodynamic coefficients for each available Seaglider fault configuration. The currently implemented CFD-derived cases include:

- Nominal configuration
- 25% wing loss
- 50% wing loss
- 50% rudder loss

## 2.3 Running the Plant

The Digital Twin function repeatedly calls the Plant dynamics code to simulate the behavior of the Seaglider throughout the dive. At each timestep, the Plant receives the current actuator state vector  $\vec{U}_i$  and current vehicle state vector  $\vec{X}_i$ , then calculates the next vehicle state vector  $\vec{X}_{i+1}$ .

After each timestep:

1. The output state vector  $\vec{X}_{i+1}$  replaces the previous state vector  $\vec{X}_i$
2. The next actuator state vector  $\vec{U}_{i+1}$  is extracted from the next row of the mission data matrix
3. The updated vectors are passed back into the Plant

This iterative update loop continues until all actuator states contained within the mission data matrix  $M$  have been processed, completing the Digital Twin simulation of the Seaglider dive.

### 3 Code

Listing 1: Digital Twin Simulation Function as of June 3rd, 2026

```
1 function Nom_Sim_Matrix = Digital_Twin(Mission_Data_Matrix, coefs,  
    params)  
2 % DIGITAL_TWIN  
3 %  
4 % Runs dynamicsV5 row-by-row using ode45.  
5 % Designed to match the dynamics_test_whole_dive workflow, but with  
6 % row-by-row controller updates from Mission_Data_Matrix.  
7 %  
8 % IMPORTANT CHANGE:  
9 %   The ODE simulation now starts only after the glider is submerged  
    past a  
10 %   selected depth threshold. Rows before that point are copied from  
    the  
11 %   mission data unchanged. This avoids trying to simulate unstable  
    surface  
12 %   startup behavior with the submerged-glider dynamics model.  
13 %  
14 % Mission data units:  
15 %   eng_depth      = cm  
16 %   vert_speed_gsm = cm/s  
17 %   density        = kg/m^3  
18 %   pressure       = pressure from NC file  
19 %  
20 % Internal dynamics units:  
21 %   depth          = m, positive downward  
22 %   vertical speed = m/s, positive downward  
23 %   density        = kg/m^3  
24 %  
25 % State vector:  
26 %   X = [phidot; thetadot; headingdot; phi; theta; heading; u; v; w;  
    x; y; z]  
27 %  
28 %   X(1:3) = Euler angle rates, deg/s  
29 %   X(4)   = roll angle, deg  
30 %   X(5)   = pitch angle, deg  
31 %   X(6)   = heading angle, deg  
32 %   X(7)   = forward/glide-path velocity, m/s  
33 %   X(8)   = lateral/glide-path velocity, m/s  
34 %   X(9)   = vertical velocity, m/s, positive downward  
35 %   X(12)  = depth, m, positive downward  
36 %  
37 %% Separate headers and numeric data  
38 headers = strtrim(string(Mission_Data_Matrix(1,:)));  
39 data = cell2mat(Mission_Data_Matrix(2:end,:));
```

```

40
41 numRows = size(data,1);
42
43 %% Find required columns
44 timeIdx = findColumn(headers, ["eng_elaps_t", "eng_elaps_t_0000"]);
45 depthIdx = findColumn(headers, ["eng_depth"]);
46 headIdx = findColumn(headers, ["eng_head"]);
47
48 pitchIdx = findColumn(headers, ["eng_pitchAng", "pitchAng", "
    pitchANG"]);
49 rollIdx = findColumn(headers, ["eng_rollAng", "rollAng", "rollANG
    "]);
50
51 pitchCtlIdx = findColumn(headers, ["eng_pitchCtl", "pitchCtl"]);
52 rollCtlIdx = findColumn(headers, ["eng_rollCtl", "rollCtl"]);
53 vbdIdx = findColumn(headers, ["eng_vbdCC", "vbdCC"]);
54
55 vertIdx = findColumn(headers, ["vert_speed_gsm"]);
56 densityIdx = findColumn(headers, ["density", "log_RHO", "$RHO"]);
57 pressureIdx = findColumn(headers, ["pressure"]);
58 tempIdx = findColumn(headers, ["temperature", "temp"]);
59
60 headingDesiredIdx = findColumn(headers, ...
61     ["$MHEAD_RNG_PITCHd_Wd", "MHEAD_RNG_PITCHd_Wd", "
        log_MHEAD_RNG_PITCHd_Wd"]);
62
63 %% Error checks
64 if isempty(timeIdx), error("Missing time column."); end
65 if isempty(depthIdx), error("Missing depth column."); end
66 if isempty(headIdx), error("Missing heading column."); end
67 if isempty(pitchIdx), error("Missing pitch angle column."); end
68 if isempty(rollIdx), error("Missing roll angle column."); end
69 if isempty(pitchCtlIdx), error("Missing pitch control column."); end
70 if isempty(rollCtlIdx), error("Missing roll control column."); end
71 if isempty(vbdIdx), error("Missing VBD column."); end
72 if isempty(vertIdx), error("Missing vertical speed column."); end
73 if isempty(densityIdx), error("Missing density column."); end
74 if isempty(pressureIdx), error("Missing pressure column."); end
75 if isempty(headingDesiredIdx), error("Missing desired heading column
    ."); end
76
77 %% Create output matrix
78 % Start by copying the mission matrix. Rows before the simulation
    start row
79 % intentionally remain unchanged.
80 Nom_Sim_Matrix = Mission_Data_Matrix;
81

```

```

82 %% Choose simulation start row
83 % The dynamics model behaves best once the glider is submerged and
    actually
84 % gliding. Starting at the surface caused the previous blow-up.
85 %
86 % Default: start once mission depth reaches 1000 cm = 10 m.
87 % To change this without editing code, add this field in
    parametersV5.m:
88 %   params.simStartDepth_cm = 500; % example: start after 5 m
89 if isfield(params, 'simStartDepth_cm')
90     simStartDepth_cm = params.simStartDepth_cm;
91 else
92     simStartDepth_cm = 1000;
93 end
94
95 startRow = find(data(:,depthIdx) >= simStartDepth_cm, 1, 'first');
96
97 if isempty(startRow)
98     warning('No depth greater than %.1f cm found. Starting
    simulation at row 1.', simStartDepth_cm);
99     startRow = 1;
100 end
101
102 fprintf('\n=====\\n');
103 fprintf('Digital Twin Simulation Start\\n');
104 fprintf('=====\\n');
105 fprintf('Simulation starts at numeric data row: %d of %d\\n',
    startRow, numRows);
106 fprintf('Mission_Data_Matrix row: %d\\n', startRow + 1);
107 fprintf('Start depth: %.2f cm\\n', data(startRow, depthIdx));
108 fprintf('Rows before this are copied unchanged from mission data.\\n'
    );
109 fprintf('=====\\n\\n');
110
111 %% Initial state vector at selected start row
112 X = zeros(12,1);
113
114 % Estimate initial angular rates near startRow.
115 % Prefer backward difference so the start state is based on the
    selected row.
116 if startRow > 1
117     ratePrevRow = startRow - 1;
118     rateNextRow = startRow;
119 elseif numRows >= 2
120     ratePrevRow = startRow;
121     rateNextRow = startRow + 1;
122 else

```

```

123     ratePrevRow = startRow;
124     rateNextRow = startRow;
125 end
126
127 dtRate = data(rateNextRow,timeIdx) - data(ratePrevRow,timeIdx);
128
129 if isfinite(dtRate) && dtRate > 0
130     X(1) = (data(rateNextRow,rollIdx) - data(ratePrevRow,rollIdx))
131           / dtRate; % phidot, deg/s
132     X(2) = (data(rateNextRow,pitchIdx) - data(ratePrevRow,pitchIdx))
133           / dtRate; % thetadot, deg/s
134
135     head1 = data(ratePrevRow,headIdx);
136     head2 = data(rateNextRow,headIdx);
137     dHead = wrapTo180_custom(head2 - head1);
138     X(3) = dHead / dtRate; % headingdot, deg/s
139 end
140
141 % Initial angles from selected mission row.
142 X(4) = data(startRow,rollIdx);
143 X(5) = data(startRow,pitchIdx);
144 X(6) = mod(data(startRow,headIdx),360);
145
146 % Initial translational velocities.
147 X(7) = 0.1; % assumed forward velocity, m/s
148 X(8) = 0; % assumed lateral velocity, m/s
149 X(9) = data(startRow,vertIdx)/100; % cm/s to m/s, positive downward
150     convention
151
152 % Initial positions.
153 X(10) = 0;
154 X(11) = 0;
155 X(12) = data(startRow,depthIdx)/100; % cm to m
156
157 % Apply initial safety cleanup.
158 X = cleanState(X);
159
160 %% Prebuild coefficient interpolants for faster dynamics calls
161 coefs = buildCoefficientInterpolants(coefs);
162
163 %% ode45 options
164 % Looser tolerances are intentional here. The digital twin is being
165     driven
166 % by 5-6 s mission samples, so very tight tolerances slow the solver
167     without
168 % improving the row-level result much.
169 odeOpts = odeset( ...

```

```

165     'RelTol', 1e-3, ...
166     'AbsTol', 1e-5, ...
167     'MaxStep', 0.5);
168
169 %% Loop through mission rows
170 % k is the current numeric mission row. The ODE integrates row k ->
    k+1.
171 for k = startRow:numRows-1
172
173     dt = data(k+1,timeIdx) - data(k,timeIdx);
174
175     fprintf('Running simulation row %d of %d\n', k, numRows-1);
176     fprintf('Row %d/%d | dt = %.3f | roll = %.2f | pitch = %.2f |
        head = %.2f | w = %.4f | depth = %.2f\n', ...
177         k, numRows-1, dt, X(4), X(5), X(6), X(9), X(12));
178
179     if ~isfinite(dt) || dt <= 0
180         warning("Bad dt at mission row %d. Holding previous state.",
            k);
181         Xnew = X;
182         Nom_Sim_Matrix = saveStateToMatrix(Nom_Sim_Matrix, k, Xnew,
            ...
183             pitchIdx, rollIdx, headIdx, vertIdx, depthIdx);
184         continue;
185     end
186
187 %% Update parameters from mission data
188 params_k = params;
189
190 params_k.heading_desired = data(k, headingDesiredIdx);
191 params_k.pressure = data(k, pressureIdx);
192 params_k.rho = data(k, densityIdx);
193
194 if ~isempty(tempIdx)
195     params_k.temp = data(k, tempIdx);
196 end
197
198 %% Build control vector from row k
199 % Pitch control appears to be cm; convert to m.
200 x_bat = data(k, pitchCtlIdx) / 100;
201
202 % Roll control is used directly as battery roll angle.
203 phi_bat = data(k, rollCtlIdx);
204
205 % VBD already in cc.
206 vbdCC = data(k, vbdIdx);
207

```

```

208     U = [x_bat; phi_bat; vbdCC];
209
210     %% Run dynamics for this row-to-row time step
211     X0 = X;
212
213     try
214         [~, Xout] = ode45(@(t,Xode) dynamicsV6_06(Xode, U, coefs,
215             params_k), ...
216             [0 dt], X0, odeOpts);
217
218         Xnew = Xout(end,:).';
219
220         if any(~isfinite(Xnew))
221             warning("Bad dynamics output at mission row %d. Holding
222                 previous state.", k);
223             Xnew = X;
224         end
225
226         fprintf('    ode45 steps used: %d\n', size(Xout,1));
227
228     catch ME
229         warning("dynamicsV5 failed at mission row %d. Holding
230             previous state. Message: %s", ...
231             k, ME.message);
232         Xnew = X;
233     end
234
235     %% Clean state values to prevent solver blow-up
236     Xnew = cleanState(Xnew);
237
238     %% Save simulated values to next mission row
239     Nom_Sim_Matrix = saveStateToMatrix(Nom_Sim_Matrix, k, Xnew, ...
240         pitchIdx, rollIdx, headIdx, vertIdx, depthIdx);
241
242     %% Carry final ODE state into next step
243     X = Xnew;
244
245 end
246
247 fprintf('\n');
248
249 %% Display summary
250 fprintf('\n===== \n');
251 fprintf('Nominal Simulation Matrix Created\n');
252 fprintf('===== \n');
253 fprintf('Rows in matrix: %d\n', numRows);

```

```

251 fprintf('Rows simulated with ODE: %d\n', max(numRows - startRow, 0))
    ;
252 fprintf('Rows copied before simulation: %d\n', startRow - 1);
253 fprintf('Updated columns after start row:\n');
254 fprintf(' - %s\n', headers(pitchIdx));
255 fprintf(' - %s\n', headers(rollIdx));
256 fprintf(' - %s\n', headers(headIdx));
257 fprintf(' - %s in cm/s\n', headers(vertIdx));
258 fprintf(' - %s in cm\n', headers(depthIdx));
259 fprintf('=====\n');
260
261 end
262
263 %% =====
264 % Helper: save state to output matrix
265 %% =====
266 function Nom_Sim_Matrix = saveStateToMatrix(Nom_Sim_Matrix, k, Xnew,
    ...
267     pitchIdx, rollIdx, headIdx, vertIdx, depthIdx)
268
269 % k is the current numeric data row.
270 % Mission_Data_Matrix has one header row, so numeric row k+1 in the
    data
271 % is matrix row k+2.
272 rowOut = k + 2;
273
274 Nom_Sim_Matrix{rowOut, pitchIdx} = Xnew(5);
275 Nom_Sim_Matrix{rowOut, rollIdx} = Xnew(4);
276 Nom_Sim_Matrix{rowOut, headIdx} = Xnew(6);
277
278 % Save internal w from m/s back to cm/s.
279 Nom_Sim_Matrix{rowOut, vertIdx} = Xnew(9) * 100;
280
281 % Convert depth from m to cm.
282 Nom_Sim_Matrix{rowOut, depthIdx} = Xnew(12) * 100;
283
284 end
285
286 %% =====
287 % Helper: clean/saturate state vector
288 %% =====
289 function X = cleanState(X)
290
291 % Remove non-finite values early.
292 if any(~isfinite(X))
293     X(~isfinite(X)) = 0;
294 end

```

```

295
296 % Heading wrap.
297 X(6) = mod(X(6),360);
298
299 % Clamp angular rates, deg/s.
300 X(1:3) = max(min(X(1:3), 20), -20);
301
302 % Clamp Euler angles.
303 % Roll is limited to +/-90 for numerical stability.
304 % Pitch is limited to +/-85 to avoid tan(theta)/sec(theta)
    singularity
305 % inside the dynamics equations.
306 X(4) = max(min(X(4), 90), -90);
307 X(5) = max(min(X(5), 85), -85);
308
309 % Clamp velocities, m/s.
310 % Seaglider vertical speeds should be far below 1 m/s, but 1 m/s
    gives
311 % enough room during debugging while preventing runaway states.
312 X(7:9) = max(min(X(7:9), 1.0), -1.0);
313
314 % Prevent negative depth.
315 if X(12) <= 0
316     X(12) = 0;
317
318     % If at surface and moving upward, stop upward motion.
319     if X(9) < 0
320         X(9) = 0;
321     end
322 end
323
324 % Hard upper bound on simulated depth for numerical protection.
325 % Increase this if your mission is deeper than 1000 m.
326 X(12) = min(X(12), 1000);
327
328 end
329
330 %% =====
331 % Helper: build coefficient interpolants
332 %% =====
333 function coefs = buildCoefficientInterpolants(coefs)
334
335 % Sort grids defensively.
336 [alphasSorted, ia] = sort(coefs.alphas(:).');
337 [betasSorted, ib] = sort(coefs.betas(:).');
338
339 coefs.alphas = alphasSorted;

```

```

340 coefs.betas = betasSorted;
341
342 coefNames = ["CLs", "CDs", "CYs", "Croll", "Cpitch", "Cyaw"];
343
344 for i = 1:numel(coefNames)
345     name = coefNames(i);
346     table = coefs.(name);
347     table = table(ia, ib);
348     coefs.(name) = table;
349 end
350
351 % Rows correspond to alpha; columns correspond to beta.
352 coefs.F_CL = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.CLs, 'linear', 'nearest');
353 coefs.F_CD = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.CDs, 'linear', 'nearest');
354 coefs.F_CY = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.CYs, 'linear', 'nearest');
355 coefs.F_Croll = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.Croll, 'linear', 'nearest');
356 coefs.F_Cpitch = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.Cpitch, 'linear', 'nearest');
357 coefs.F_Cyaw = griddedInterpolant({coefs.alphas, coefs.betas},
    coefs.Cyaw, 'linear', 'nearest');
358
359 end
360
361 %% =====
362 % Helper: find column
363 %% =====
364 function idx = findColumn(headers, possibleNames)
365
366 idx = [];
367
368 for i = 1:length(possibleNames)
369     idx = find(strcmp(headers, possibleNames(i)), 1);
370
371     if ~isempty(idx)
372         return;
373     end
374 end
375
376 end
377
378 %% =====
379 % Helper: wrap angle to [-180, 180]
380 %% =====

```

```

381 function angleOut = wrapTo180_custom(angleIn)
382
383 angleOut = mod(angleIn + 180, 360) - 180;
384
385 end

```

## 4 Code Explanation per Section

### 4.1 Separate Headers and Numeric Data

This section separates the mission data matrix into two components: the header row and the numeric data. The header row contains the names of each variable (e.g., `eng_depth`, `eng_head`, `eng_pitchAng`) and is stored separately for later column identification. The remaining rows are converted into a purely numeric matrix to allow efficient mathematical operations and simulation calculations.

The total number of mission samples is also determined and stored for use throughout the simulation.

### 4.2 Find Required Columns

This section identifies the location of all variables required by the Digital Twin. Since mission files may have different column ordering, the code searches for variables by name rather than relying on fixed column numbers.

Examples of identified variables include:

- Time
- Depth
- Heading
- Pitch Angle
- Roll Angle
- Pitch Controller Position
- Roll Controller Position
- Variable Buoyancy Device (VBD)
- Vertical Velocity
- Density
- Pressure
- Desired Heading

This approach increases robustness and allows the code to operate on different mission datasets without modification.

### 4.3 Error Checks

This section verifies that all required columns were successfully located. If a required variable is missing, the simulation immediately terminates and provides a descriptive error message indicating which variable could not be found.

Performing these checks early prevents difficult-to-diagnose failures later in the simulation.

### 4.4 Create Output Matrix

An output matrix, `Nom_Sim_Matrix`, is created by copying the original mission data matrix.

Initially, this matrix is identical to the mission data. As the simulation progresses, selected variables are replaced with Digital Twin outputs while all other mission data remains unchanged.

### 4.5 Choose Simulation Start Row

The Digital Twin is designed to model submerged glider dynamics. Surface behavior can be difficult to model accurately and often causes numerical instability.

To avoid these issues, the simulation begins only after the glider reaches a specified depth threshold. The default threshold is:

$$1000 \text{ cm} = 10 \text{ m}$$

All mission rows prior to this depth are copied directly into the output matrix without modification.

### 4.6 Initialize State Vector

This section constructs the initial 12-state vector used by the dynamics model:

$$X = \begin{bmatrix} \dot{\phi} & \dot{\theta} & \dot{\psi} & \phi & \theta & \psi & u & v & w & x \\ y & z & & & & & & & & \end{bmatrix}^T$$

where:

- $\phi$  = Roll Angle
- $\theta$  = Pitch Angle
- $\psi$  = Heading Angle
- $u$  = Forward Velocity
- $v$  = Lateral Velocity

- $w$  = Vertical Velocity
- $z$  = Depth

Angular rates are estimated using finite differences between nearby mission samples. Initial angles, depth, and vertical velocity are extracted directly from the mission data at the selected simulation start row.

## 4.7 Prebuild Coefficient Interpolants

Hydrodynamic coefficients such as lift, drag, side force, and moment coefficients are stored as lookup tables.

This section converts the lookup tables into interpolation functions that can rapidly return coefficient values during simulation. Creating these interpolants once at the beginning significantly improves runtime performance compared to repeatedly performing interpolation inside the dynamics model.

## 4.8 ODE45 Solver Options

The numerical integration settings for `ode45` are defined in this section.

Key settings include:

- Relative tolerance
- Absolute tolerance
- Maximum step size

These settings balance simulation accuracy and computational efficiency.

## 4.9 Loop Through Mission Rows

This is the primary simulation loop.

For each mission row, the Digital Twin integrates the vehicle dynamics from the current mission sample to the next mission sample.

The simulation progresses sequentially through the mission, updating the state vector after each integration step.

## 4.10 Update Parameters From Mission Data

Environmental conditions can vary throughout the mission.

This section updates the simulation parameters using the current mission row, including:

- Water density
- Pressure

- Temperature
- Desired heading

Using real mission conditions improves simulation fidelity.

#### 4.11 Build Control Vector

The control inputs commanded by the glider are extracted from the mission data and assembled into a control vector:

$$U = \begin{bmatrix} x_{bat} \\ \phi_{bat} \\ VBD \end{bmatrix}$$

where:

- $x_{bat}$  = Pitch battery position
- $\phi_{bat}$  = Roll battery position
- VBD = Variable Buoyancy Device volume

These values represent the glider control actions used by the dynamics model.

#### 4.12 Run Dynamics Simulation

The Digital Twin calls `ode45` to integrate the equations of motion over the current mission timestep.

The dynamics function computes the state derivatives according to:

$$\dot{X} = f(X, U)$$

where  $X$  is the current state vector and  $U$  is the control vector.

The final state returned by the integrator becomes the predicted vehicle state at the next mission sample.

#### 4.13 Clean State Values

Numerical simulations can occasionally produce unrealistic values due to solver instability or unusual operating conditions.

This section applies safety limits to the state vector, including:

- Heading normalization
- Roll angle limits
- Pitch angle limits

- Velocity limits
- Non-negative depth enforcement

These limits prevent numerical blow-up and maintain physically realistic simulation results.

#### **4.14 Save Simulated Values**

The newly calculated simulation outputs are written into the output matrix.

The following variables are updated:

- Pitch Angle
- Roll Angle
- Heading
- Vertical Velocity
- Depth

Unit conversions are performed where necessary before storing the values.

#### **4.15 Carry Final State Forward**

The final state computed by the ODE solver becomes the starting state for the next simulation step.

This allows the Digital Twin to continuously evolve throughout the mission rather than restarting from the same initial condition at every row.

#### **4.16 Display Simulation Summary**

After all mission rows have been processed, a summary is displayed showing:

- Total rows in the mission
- Number of rows simulated
- Number of rows copied directly from mission data
- Variables updated by the Digital Twin

This provides a quick verification that the simulation completed successfully.

#### **4.17 Helper Function: Save State to Matrix**

This helper function writes simulated states back into the output matrix and performs necessary unit conversions before storage.

## 4.18 Helper Function: Clean State

This helper function enforces numerical stability by removing invalid values and constraining states to physically realistic ranges.

## 4.19 Helper Function: Build Coefficient Interpolants

This helper function creates interpolation objects for lift, drag, side force, and moment coefficient lookup tables, enabling rapid coefficient evaluation during simulation.

## 4.20 Helper Function: Find Column

This helper function searches the mission data headers for a specified variable name and returns the corresponding column index.

## 4.21 Helper Function: Wrap Angle to $[-180^\circ, 180^\circ]$

This helper function converts any heading angle into the range:

$$-180^\circ \leq \psi \leq 180^\circ$$

This ensures heading differences are computed correctly and prevents discontinuities when crossing the  $0^\circ/360^\circ$  boundary.

# 5 Digital Twin Initial Design

## 5.1 Introduction

The first iteration of the Digital Twin was designed as a fully independent simulation of Seaglider behavior that did not rely on mission data received after a dive, such as the `.eng`, `.nc`, and capture files. The goal of this approach was to predict the vehicle’s behavior using only mission commands and an internal model of the glider itself. To achieve this, the DT included models of the pitch battery, roll battery, and VBD actuations, as well as an energy model to estimate the power consumed by these movements throughout a dive.

A key challenge of this design stemmed from the Seaglider’s controller, which continuously determines whether the vehicle is on course and commands the appropriate actuator movements to correct its trajectory when necessary. Unfortunately, the controller logic is proprietary and protected under ITAR regulations, making it inaccessible to the capstone team. As a result, recreating the controller’s decision-making process became one of the most challenging aspects of the DT design.

The complexity associated with approximating the controller behavior ultimately led the 2026 Capstone Team to pivot toward the final DT architecture. In the final DT design, actuator states are obtained directly from the transmitted `.nc` and `.log` files rather than being independently modeled. This significantly reduced uncertainty in the simulation and simplified the overall architecture. The tradeoff, however, is that the DT now depends on the

Seaglider successfully completing a dive and transmitting its data. As a result, the D.A.P system operates using information from the previous dive, effectively performing its analysis one dive behind the vehicle.

The following sections describe the original Digital Twin architecture, including the waypoint-based guidance strategy developed to approximate the unavailable controller logic, along with a breakdown of the proposed subsystems and their interactions.

## 5.2 Waypoint-Based Guidance Logic

One of the primary challenges in the initial Digital Twin architecture was replicating the Seaglider’s closed-loop navigation behavior without access to the proprietary controller logic. To approximate the vehicle’s decision-making process, a waypoint-based guidance strategy was developed. This approach discretizes a dive profile into a series of reference positions that the Digital Twin attempts to reach sequentially throughout the simulation.

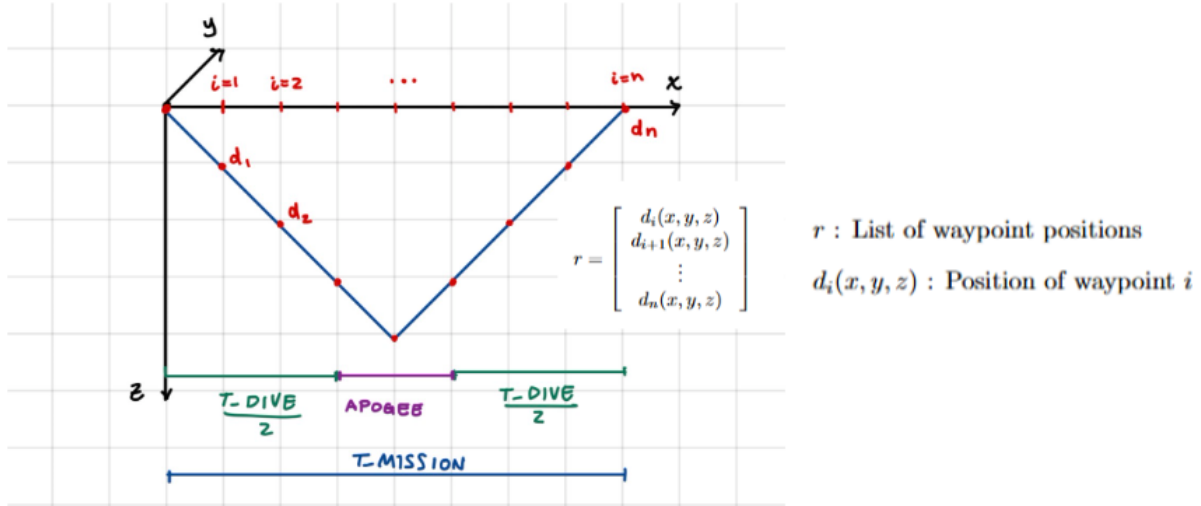


Figure 2: Waypoint segmentation strategy used by the Digital Twin.

Figure 2 illustrates the waypoint generation concept. Mission data collected from a completed dive is used to generate an idealized dive trajectory. Following the SSC log parameter notation, the total mission duration is denoted as  $T\_MISSION$ , while the dive and climb phases are represented by  $\frac{T-DIVE}{2}$  on either side of the apogee.

The reference trajectory is divided into discrete intervals, creating a sequence of waypoints:

$$r = \begin{bmatrix} d_1(x, y, z) \\ d_2(x, y, z) \\ \vdots \\ d_n(x, y, z) \end{bmatrix}$$

where  $d_i(x, y, z)$  represents the three-dimensional position of waypoint  $i$ .

The collection of waypoints, denoted by  $r$ , serves as the primary input to the Digital Twin. During execution, the simulated vehicle progresses through the waypoint list sequentially. Once a waypoint is reached, the next waypoint becomes the active navigation target. In this manner, the waypoint list acts as a surrogate for the unavailable controller logic by providing intermediate reference positions throughout the dive profile.

### 5.3 Digital Twin System Architecture

The overall architecture of the original Digital Twin concept is shown in Figure 3. The design was intended to operate as a fully independent simulation capable of predicting Seaglider behavior from mission commands alone.

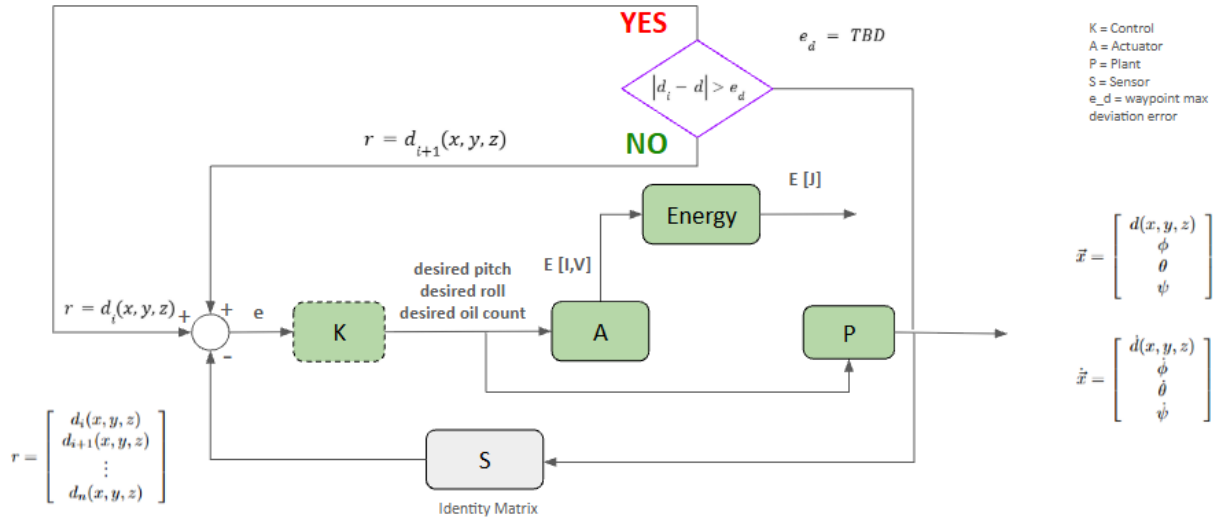


Figure 3: Original Digital Twin architecture incorporating waypoint navigation and actuator state modeling.

The simulation begins by selecting the current reference waypoint,

$$r = d_i(x, y, z),$$

from the waypoint list generated during mission preprocessing. The difference between the reference waypoint and the current simulated vehicle position is used to calculate a position error,

$$e = d_i(x, y, z) - d(x, y, z).$$

This error is supplied to the controller block, denoted as  $K$ . The purpose of the controller is to determine the actuator commands necessary to guide the vehicle toward the desired waypoint. In the physical Seaglider, this controller is responsible for maintaining course and correcting deviations throughout the dive. However, because the controller implementation is proprietary and unavailable to the project team, its behavior could only be approximated.

The controller outputs desired actuator commands including:

- Desired pitch battery position
- Desired roll battery position
- Desired VBD position

These commands are sent to the actuator subsystem, represented by block *A*. Historical mission data was investigated to identify relationships between commanded vehicle behavior and resulting actuator movements. The objective was to estimate actuator states from mission conditions and controller outputs.

The actuator subsystem produces the physical movements of the internal battery packs and VBD mechanism. These movements require electrical power and therefore feed into the energy model.

### 5.3.1 Energy Model

The Energy subsystem estimates the electrical energy consumed by actuator operations. Using predicted actuator motions, the model calculates the required current and voltage associated with battery translation and VBD actuation.

The resulting energy consumption is accumulated and reported in Joules,

$$E [J],$$

providing an estimate of the energy expenditure associated with a simulated dive profile.

### 5.3.2 Plant Dynamics Model

The actuator outputs are simultaneously provided to the Plant block, denoted as *P*. The Plant represents the vehicle dynamics model and calculates the resulting forces, moments, and motion generated by the actuator commands.

The Plant outputs the simulated state vector,

$$\mathbf{x} = \begin{bmatrix} d(x, y, z) \\ \phi \\ \theta \\ \psi \end{bmatrix},$$

along with the corresponding linear and angular velocity states.

These outputs represent the Digital Twin's estimate of the vehicle's position, orientation, and dynamic behavior throughout the dive.

### 5.3.3 Sensor Model

The Sensor block, denoted as *S*, was included to maintain consistency with traditional control-system architectures. However, because no sensor error modeling was planned for this version of the Digital Twin, the Sensor subsystem was implemented as an identity matrix. Consequently, the state vector produced by the Plant is passed directly back into the control loop without modification.

$$S = I$$

where  $I$  is the identity matrix.

## 5.4 Waypoint Transition Logic

A key component of the architecture is determining when the simulated vehicle has successfully reached a waypoint. To accomplish this, a waypoint acceptance threshold,  $e_d$ , is defined.

The distance between the current reference waypoint and the simulated vehicle position is continuously evaluated:

$$|d_i - d|.$$

If the distance remains greater than the allowable waypoint tolerance,

$$|d_i - d| > e_d,$$

the waypoint has not yet been reached and the current waypoint remains the active target.

If the distance falls below the waypoint tolerance,

$$|d_i - d| < e_d,$$

the waypoint is considered reached and the Digital Twin advances to the next waypoint in the reference list:

$$r = d_{i+1}(x, y, z).$$

This decision process is represented by the diamond-shaped logic block shown in Figure 3. The waypoint update mechanism was developed as an attempt to replicate the corrective navigation decisions that would normally be performed by the Seaglider’s proprietary controller. By continuously advancing through the waypoint sequence, the Digital Twin could approximate trajectory tracking despite lacking direct access to the vehicle’s internal guidance algorithms.

In the next section all blocks building the Digital Twin architecture are discussed in detail.

## 5.5 Inputs and Outputs to Actuators and Plant

Figure 4 shows the inputs and outputs to the actuator blocks and plant. As discussed above, both actuators calculated the actuator movement and associated energy consumption. The outputs of these actuator models were then passed through the power modeler, which estimated the total energy usage throughout the mission. The resulting actuator states were then used to calculate vehicle forces and moments before being passed into the dynamics model to predict vehicle states and motions.

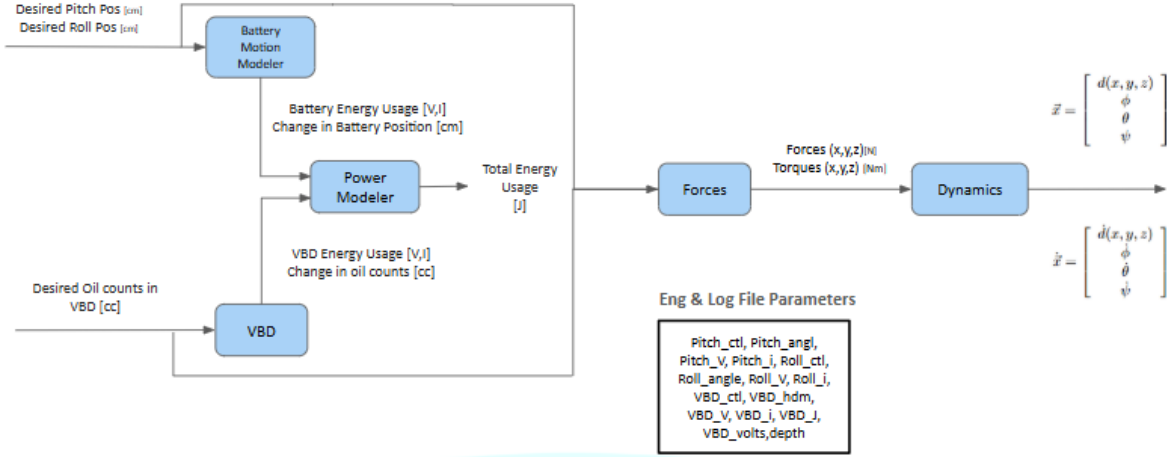


Figure 4: Old Version of the Digital Twin Block Diagram

### 5.5.1 Battery Motion and Energy Modeling

One of the most significant features of the preliminary design was the inclusion of a Battery Motion Modeler. Rather than treating pitch and roll commands as direct inputs to the dynamics equations, the model simulated the physical movement of the Seaglider’s internal battery pack. Changes in battery position altered the vehicle center of gravity, generating pitching and rolling moments that influenced the vehicle’s attitude. Simultaneously, the model tracked the electrical energy required to move the battery between commanded positions. By accounting for actuator energy consumption, the Digital Twin could estimate the operational cost of attitude corrections and provide insight into how control actions affected overall mission endurance. This capability was removed in the current implementation, where battery position is treated primarily as a control input contained within the actuator state vector rather than as a subsystem with its own power and motion dynamics.

### 5.5.2 Variable Bouyancy Device Modeling

The preliminary design also contained a dedicated VBD subsystem that modeled changes in oil volume within the Variable Buoyancy Device. The VBD model calculated both the commanded buoyancy adjustment and the energy required to pump oil during operation. These changes directly influenced vehicle buoyancy forces before being passed to the dynamics model. This approach allowed the Digital Twin to estimate the energetic cost of depth control and glide-path adjustments. In the current Digital Twin, the VBD state is extracted directly from the mission data matrix and incorporated into the control vector used by the dynamics model. While this significantly simplifies implementation, it removes the ability to evaluate how future control actions may impact battery reserves or mission duration.

### 5.5.3 Power Modeler Removal

A major architectural change between the two versions is the removal of the Power Model. The preliminary design explicitly calculated total energy consumption by combining bat-

tery actuator energy usage and VBD energy usage. This information could have been used to evaluate mission endurance, estimate remaining energy reserves, or support mitigation strategies that balanced fault recovery with power availability. The current Digital Twin no longer includes an energy tracking capability and instead focuses solely on reproducing vehicle dynamics and comparing simulated outputs to mission data for fault detection and diagnosis. This change reduced model complexity and computational requirements but eliminated the ability to assess vehicle performance from an energy management perspective.

#### **5.5.4 Transition to the Current Design**

The current Digital Twin architecture was redesigned to prioritize accurate reconstruction of Seaglider motion and compatibility with the D.A.P. fault detection framework. Rather than modeling individual actuator mechanisms and energy consumption, the simulation now uses recorded mission actuator states, environmental conditions, vehicle parameters, and hydrodynamic coefficients as direct inputs to a dynamics-based Plant model. The simulation iteratively propagates vehicle states throughout the dive and produces estimates of orientation, angular rates, velocity, and position that can be compared against recorded mission data to identify faults or performance deviations. This streamlined architecture reduced the number of subsystem models required while improving maintainability and reducing computational overhead.

#### **5.5.5 Design Tradeoff Discussion**

The preliminary architecture offered greater physical fidelity by modeling actuator behavior and power consumption, making it potentially useful for future fault mitigation and mission planning applications. However, it required additional subsystem development, actuator characterization, and power modeling that increased implementation complexity. The current architecture sacrifices energy-awareness in favor of a more focused dynamics simulation capable of supporting fault detection and diagnostic objectives. As a result, the present Digital Twin is better suited for comparing simulated and recorded vehicle behavior, while the preliminary design provided a foundation for future expansion into energy management, mission endurance prediction, and autonomous fault recovery strategies.